

DEFINITION 7.12

P is the class of languages that are decidable in polynomial time on a deterministic single-tape Turing machine. In other words,

$$P = \bigcup_k \text{TIME}(n^k).$$

The class P plays a central role in our theory and is important because

1. P is invariant for all models of computation that are polynomially equivalent to the deterministic single-tape Turing machine, and
2. P roughly corresponds to the class of problems that are realistically solvable on a computer.

Item 1 indicates that P is a mathematically robust class. It isn't affected by the particulars of the model of computation that we are using.

Item 2 indicates that P is relevant from a practical standpoint. When a problem is in P , we have a method of solving it that runs in time n^k for some constant k . Whether this running time is practical depends on k and on the application. Of course, a running time of n^{100} is unlikely to be of any practical use. Nevertheless, calling polynomial time the threshold of practical solvability has proven to be useful. Once a polynomial time algorithm has been found for a problem that formerly appeared to require exponential time, some key insight into it has been gained and further reductions in its complexity usually follow, often to the point of actual practical utility.

EXAMPLES OF PROBLEMS IN P

When we present a polynomial time algorithm, we give a high-level description of it without reference to features of a particular computational model. Doing so avoids tedious details of tapes and head motions. We follow certain conventions when describing an algorithm so that we can analyze it for polynomiality.

We continue to describe algorithms with numbered stages. Now we must be sensitive to the number of Turing machine steps required to implement each stage, as well as to the total number of stages that the algorithm uses.

When we analyze an algorithm to show that it runs in polynomial time, we need to do two things. First, we have to give a polynomial upper bound (usually in big- O notation) on the number of stages that the algorithm uses when it runs on an input of length n . Then, we have to examine the individual stages in the description of the algorithm to be sure that each can be implemented in polynomial time on a reasonable deterministic model. We choose the stages when we describe the algorithm to make this second part of the analysis easy to do. When both tasks have been completed, we can conclude that the algorithm

THEOREM

7.14

 $PATH \in P$.

PROOF IDEA We prove this theorem by presenting a polynomial time algorithm that decides $PATH$. Before describing that algorithm, let's observe that a brute-force algorithm for this problem isn't fast enough.

A brute-force algorithm for $PATH$ proceeds by examining all potential paths in G and determining whether any is a directed path from s to t . A potential path is a sequence of nodes in G having a length of at most m , where m is the number of nodes in G . (If any directed path exists from s to t , one having a length of at most m exists because repeating a node never is necessary.) But the number of such potential paths is roughly m^m , which is exponential in the number of nodes in G . Therefore, this brute-force algorithm uses exponential time.

To get a polynomial time algorithm for $PATH$, we must do something that avoids brute force. One way is to use a graph-searching method such as breadth-first search. Here, we successively mark all nodes in G that are reachable from s by directed paths of length 1, then 2, then 3, through m . Bounding the running time of this strategy by a polynomial is easy.

PROOF A polynomial time algorithm M for $PATH$ operates as follows.

$M =$ "On input $\langle G, s, t \rangle$, where G is a directed graph with nodes s and t :

1. Place a mark on node s .
2. Repeat the following until no additional nodes are marked:
3. Scan all the edges of G . If an edge (a, b) is found going from a marked node a to an unmarked node b , mark node b .
4. If t is marked, *accept*. Otherwise, *reject*."

Now we analyze this algorithm to show that it runs in polynomial time. Obviously, stages 1 and 4 are executed only once. Stage 3 runs at most m times because each time except the last it marks an additional node in G . Thus, the total number of stages used is at most $1 + 1 + m$, giving a polynomial in the size of G .

Stages 1 and 4 of M are easily implemented in polynomial time on any reasonable deterministic model. Stage 3 involves a scan of the input and a test of whether certain nodes are marked, which also is easily implemented in polynomial time. Hence M is a polynomial time algorithm for $PATH$.

Let's turn to another example of a polynomial time algorithm. Say that two numbers are *relatively prime* if 1 is the largest integer that evenly divides them both. For example, 10 and 13

both are divisible by 2. Let *RELPRIME* be the problem of testing whether two numbers are relatively prime. Thus

$$\text{RELPRIME} = \{ \langle x, y \rangle \mid x \text{ and } y \text{ are relatively prime} \}.$$

THEOREM 7.15

RELPRIME $\in$ P.

PROOF IDEA One algorithm that solves this problem searches through all possible divisors of both numbers and accepts if none are greater than 1. However, the magnitude of a number represented in binary, or in any other base k notation for $k \geq 2$, is exponential in the length of its representation. Therefore, this brute-force algorithm searches through an exponential number of potential divisors and has an exponential running time.

Instead, we solve this problem with an ancient numerical procedure, called the *Euclidean algorithm*, for computing the greatest common divisor. The *greatest common divisor* of natural numbers x and y , written $\text{gcd}(x, y)$, is the largest integer that evenly divides both x and y . For example, $\text{gcd}(18, 24) = 6$. Obviously, x and y are relatively prime iff $\text{gcd}(x, y) = 1$. We describe the Euclidean algorithm as algorithm E in the proof. It uses the mod function, where $x \bmod y$ is the remainder after the integer division of x by y .

PROOF The Euclidean algorithm E is as follows.

$E =$ "On input $\langle x, y \rangle$, where x and y are natural numbers in binary:

1. Repeat until $y = 0$:
2. Assign $x \leftarrow x \bmod y$.
3. Exchange x and y .
4. Output x ."

Algorithm R solves *RELPRIME*, using E as a subroutine.

$R =$ "On input $\langle x, y \rangle$, where x and y are natural numbers in binary:

1. Run E on $\langle x, y \rangle$.
2. If the result is 1, *accept*. Otherwise, *reject*."

Clearly, if E runs correctly in polynomial time, so does R and hence we only need to analyze E for time and correctness. The correctness of this algorithm is well known so we won't discuss it further here.

To analyze the time complexity of E , we first show that every execution of stage 2 (except possibly the first) cuts the value of x by at least half. After stage 2 is executed, $x < y$ because of the nature of the mod function. After stage 3, $x > y$ because the two have been exchanged. Thus, when stage 2 is subsequently

executed, $x > y$. If $x/2 \geq y$, then $x \bmod y < y \leq x/2$ and x drops by at least half. If $x/2 < y$, then $x \bmod y = x - y < x/2$ and x drops by at least half.

The values of x and y are exchanged every time stage 3 is executed, so each of the original values of x and y are reduced by at least half every other time through the loop. Thus, the maximum number of times that stages 2 and 3 are executed is the lesser of $2 \log_2 x$ and $2 \log_2 y$. These logarithms are proportional to the lengths of the representations, giving the number of stages executed as $O(n)$. Each stage of E uses only polynomial time, so the total running time is polynomial.

The final example of a polynomial time algorithm shows that every context-free language is decidable in polynomial time.

THEOREM 7.16

Every context-free language is a member of P.

PROOF IDEA In Theorem 4.9, we proved that every CFL is decidable. To do so, we gave an algorithm for each CFL that decides it. If that algorithm runs in polynomial time, the current theorem follows as a corollary. Let's recall that algorithm and find out whether it runs quickly enough.

Let L be a CFL generated by CFG G that is in Chomsky normal form. From Problem 2.26, any derivation of a string w has $2n - 1$ steps, where n is the length of w because G is in Chomsky normal form. The decider for L works by trying all possible derivations with $2n - 1$ steps when its input is a string of length n . If any of these is a derivation of w , the decider accepts; if not, it rejects.

A quick analysis of this algorithm shows that it doesn't run in polynomial time. The number of derivations with k steps may be exponential in k , so this algorithm may require exponential time.

To get a polynomial time algorithm, we introduce a powerful technique called *dynamic programming*. This technique uses the accumulation of information about smaller subproblems to solve larger problems. We record the solution to any subproblem so that we need to solve it only once. We do so by making a table of all subproblems and entering their solutions systematically as we find them.

In this case, we consider the subproblems of determining whether each variable in G generates each substring of w . The algorithm enters the solution to this subproblem in an $n \times n$ table. For $i \leq j$, the (i, j) th entry of the table contains the collection of variables that generate the substring $w_i w_{i+1} \cdots w_j$. For $i > j$, the table entries are unused.

The algorithm fills in the table entries for each substring of w . First it fills in the entries for the substrings of length 1, then those of length 2, and so on.

It uses the entries for the shorter lengths to assist in determining the entries for the longer lengths.

For example, suppose that the algorithm has already determined which variables generate all substrings up to length k . To determine whether a variable A generates a particular substring of length $k+1$, the algorithm splits that substring into two nonempty pieces in the k possible ways. For each split, the algorithm examines each rule $A \rightarrow BC$ to determine whether B generates the first piece and C generates the second piece, using table entries previously computed. If both B and C generate the respective pieces, A generates the substring and so is added to the associated table entry. The algorithm starts the process with the strings of length 1 by examining the table for the rules $A \rightarrow b$.

PROOF The following algorithm D implements the proof idea. Let G be a CFG in Chomsky normal form generating the CFL L . Assume that S is the start variable. (Recall that the empty string is handled specially in a Chomsky normal form grammar. The algorithm handles the special case in which $w = \epsilon$ in stage 1.) Comments appear inside double brackets.

$D =$ "On input $w = w_1 \cdots w_n$:

1. For $w = \epsilon$, if $S \rightarrow \epsilon$ is a rule, *accept*; else, *reject*. [$w = \epsilon$ case]
2. For $i = 1$ to n : [examine each substring of length 1]
3. For each variable A :
4. Test whether $A \rightarrow b$ is a rule, where $b = w_i$.
5. If so, place A in $table(i, i)$.
6. For $l = 2$ to n : [l is the length of the substring]
7. For $i = 1$ to $n - l + 1$: [i is the start position of the substring]
8. Let $j = i + l - 1$. [j is the end position of the substring]
9. For $k = i$ to $j - 1$: [k is the split position]
10. For each rule $A \rightarrow BC$:
11. If $table(i, k)$ contains B and $table(k + 1, j)$ contains C , put A in $table(i, j)$.
12. If S is in $table(1, n)$, *accept*; else, *reject*."

Now we analyze D . Each stage is easily implemented to run in polynomial time. Stages 4 and 5 run at most nv times, where v is the number of variables in G and is a fixed constant independent of n ; hence these stages run $O(n)$ times. Stage 6 runs at most n times. Each time stage 6 runs, stage 7 runs at most n times. Each time stage 7 runs, stages 8 and 9 run at most n times. Each time stage 9 runs, stage 10 runs r times, where r is the number of rules of G and is another fixed constant. Thus stage 11, the inner loop of the algorithm, runs $O(n^3)$ times. Summing the total shows that D executes $O(n^3)$ stages.

DEFINITION 7.19

NP is the class of languages that have polynomial time verifiers.

The class NP is important because it contains many problems of practical interest. From the preceding discussion, both *HAMPATH* and *COMPOSITES* are members of NP. As we mentioned, *COMPOSITES* is also a member of P, which is a subset of NP; but proving this stronger result is much more difficult. The term NP comes from *nondeterministic polynomial time* and is derived from an alternative characterization by using nondeterministic polynomial time Turing machines. Problems in NP are sometimes called NP-problems.

The following is a nondeterministic Turing machine (NTM) that decides the *HAMPATH* problem in nondeterministic polynomial time. Recall that in Definition 7.9, we defined the time of a nondeterministic machine to be the time used by the longest computation branch.

$N_1 =$ "On input (G, s, t) , where G is a directed graph with nodes s and t :

1. Write a list of m numbers, $p_1, \dots, p_m$, where m is the number of nodes in G . Each number in the list is nondeterministically selected to be between 1 and m .
2. Check for repetitions in the list. If any are found, *reject*.
3. Check whether $s = p_1$ and $t = p_m$. If either fail, *reject*.
4. For each i between 1 and $m - 1$, check whether (p_i, p_{i+1}) is an edge of G . If any are not, *reject*. Otherwise, all tests have been passed, so *accept*."

To analyze this algorithm and verify that it runs in nondeterministic polynomial time, we examine each of its stages. In stage 1, the nondeterministic selection clearly runs in polynomial time. In stages 2 and 3, each part is a simple check, so together they run in polynomial time. Finally, stage 4 also clearly runs in polynomial time. Thus, this algorithm runs in nondeterministic polynomial time.

THEOREM 7.20

A language is in NP iff it is decided by some nondeterministic polynomial time Turing machine.

PROOF IDEA We show how to convert a polynomial time verifier to an equivalent polynomial time NTM and vice versa. The NTM simulates the verifier by guessing the certificate. The verifier simulates the NTM by using the accepting branch as the certificate.

PROOF For the forward direction of this theorem, let $A \in \text{NP}$ and show that A is decided by a polynomial time NTM N . Let V be the polynomial time verifier for A that exists by the definition of NP. Assume that V is a TM that runs in time n^k and construct N as follows.

- $N =$ "On input w of length n :
1. Nondeterministically select string c of length at most n^k .
 2. Run V on input $\langle w, c \rangle$.
 3. If V accepts, accept; otherwise, reject."

To prove the other direction of the theorem, assume that A is decided by a polynomial time NTM N and construct a polynomial time verifier V as follows.

- $V =$ "On input $\langle w, c \rangle$, where w and c are strings:
1. Simulate N on input w , treating each symbol of c as a description of the nondeterministic choice to make at each step (as in the proof of Theorem 3.16).
 2. If this branch of N 's computation accepts, accept; otherwise, reject."

We define the nondeterministic time complexity class $\text{NTIME}(t(n))$ as analogous to the deterministic time complexity class $\text{TIME}(t(n))$.

DEFINITION 7.21

$\text{NTIME}(t(n)) = \{L \mid L \text{ is a language decided by an } O(t(n)) \text{ time nondeterministic Turing machine}\}.$

COROLLARY 7.22

$$\text{NP} = \bigcup_k \text{NTIME}(n^k).$$

The class NP is insensitive to the choice of reasonable nondeterministic computational model because all such models are polynomially equivalent. When describing and analyzing nondeterministic polynomial time algorithms, we follow the preceding conventions for deterministic polynomial time algorithms. Each stage of a nondeterministic polynomial time algorithm must have an obvious implementation in nondeterministic polynomial time on a reasonable nondeterministic computational model. We analyze the algorithm to show that every branch uses at most polynomially many stages.

EXAMPLES OF PROBLEMS IN NP

A *clique* in an undirected graph is a subgraph, wherein every two nodes are connected by an edge. A *k-clique* is a clique that contains k nodes. Figure 7.23 illustrates a graph with a 5-clique.

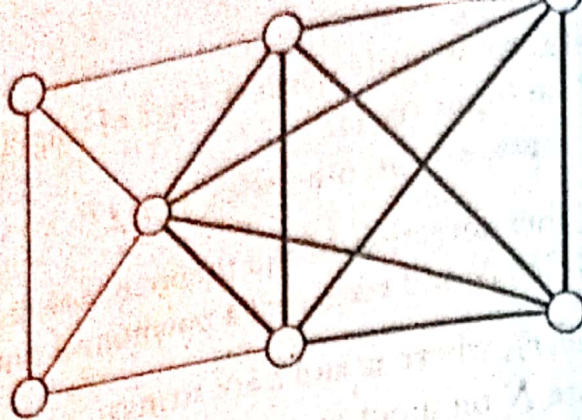


FIGURE 7.23
A graph with a 5-clique

The clique problem is to determine whether a graph contains a clique of specified size. Let

$$\text{CLIQUE} = \{ \langle G, k \rangle \mid G \text{ is an undirected graph with a } k\text{-clique} \}.$$

THEOREM

7.24

CLIQUE is in NP.

PROOF IDEA The clique is the certificate.

PROOF The following is a verifier V for CLIQUE.

$V =$ "On input $\langle \langle G, k \rangle, c \rangle$:

1. Test whether c is a subgraph with k nodes in G .
2. Test whether G contains all edges connecting nodes in c .
3. If both pass, *accept*; otherwise, *reject*."

ALTERNATIVE PROOF If you prefer to think of NP in terms of nondeterministic polynomial time Turing machines, you may prove this theorem giving one that decides CLIQUE. Observe the similarity between the two problems.

$N =$ "On input $\langle G, k \rangle$, where G is a graph:

1. Nondeterministically select a subset c of k nodes of G .
2. Test whether G contains all edges connecting nodes in c .
3. If yes, *accept*; otherwise, *reject*."

Next, we consider the SUBSET-SUM problem concerning integer arithmetic. We are given a collection of numbers $x_1, \dots, x_k$ and a target number t . We want to determine whether the collection contains a subcollection that adds up to t .

Thus,

$SUBSET-SUM = \{ \langle S, t \rangle \mid S = \{x_1, \dots, x_k\}, \text{ and for some } \{y_1, \dots, y_m\} \subseteq \{x_1, \dots, x_k\}, \text{ we have } \sum y_i = t \}$

For example, $\langle \{4, 11, 16, 21, 27\}, 25 \rangle \in SUBSET-SUM$ because $4 + 21 = 25$.
Note that $\{x_1, \dots, x_k\}$ and $\{y_1, \dots, y_m\}$ are considered to be multisets and so allow repetition of elements.

THEOREM 7.25

$SUBSET-SUM$ is in NP.

PROOF IDEA The subset is the certificate.

PROOF The following is a verifier V for $SUBSET-SUM$.

$V =$ "On input $\langle \langle S, t \rangle, c \rangle$:

1. Test whether c is a collection of numbers that sum to t .
2. Test whether S contains all the numbers in c .
3. If both pass, *accept*; otherwise, *reject*."

ALTERNATIVE PROOF We can also prove this theorem by giving a nondeterministic polynomial time Turing machine for $SUBSET-SUM$ as follows.

$N =$ "On input $\langle S, t \rangle$:

1. Nondeterministically select a subset c of the numbers in S .
2. Test whether c is a collection of numbers that sum to t .
3. If the test passes, *accept*; otherwise, *reject*."

Observe that the complements of these sets, $\overline{CLIQUE}$ and $\overline{SUBSET-SUM}$, are not obviously members of NP. Verifying that something is *not* present seems to be more difficult than verifying that it *is* present. We make a separate complexity class, called coNP, which contains the languages that are complements of languages in NP. We don't know whether coNP is different from NP.

THE P VERSUS NP QUESTION

As we have been saying, NP is the class of languages that are solvable in polynomial time on a nondeterministic Turing machine; or, equivalently, it is the class of languages whereby membership in the language can be verified in polynomial

form. A *literal* is a Boolean variable or a negated Boolean variable, as in x_i or $\neg x_i$. A *clause* is several literals connected with $\vee$ s, as in $(x_1 \vee \neg x_2 \vee \neg x_3)$. A *3cnf-formula* is in *conjunctive normal form*, called a *cnf-formula*, if it is a conjunction of several clauses connected with $\wedge$ s, as in

$$(x_1 \vee \neg x_2 \vee \neg x_3) \wedge (x_1 \vee \neg x_4 \vee x_5) \wedge (x_2 \vee \neg x_6).$$

It is a *3cnf-formula* if all the clauses have three literals, as in

$$(x_1 \vee \neg x_2 \vee \neg x_3) \wedge (x_4 \vee \neg x_5 \vee x_6) \wedge (x_2 \vee \neg x_7 \vee x_8) \wedge (x_4 \vee x_9 \vee x_{10}).$$

Let $3SAT = \{(\phi) \mid \phi \text{ is a satisfiable 3cnf-formula}\}$. If an assignment satisfies a *cnf-formula*, each clause must contain at least one literal that evaluates to 1.

The following theorem presents a polynomial time reduction from the *3SAT* problem to the *CLIQUE* problem.

THEOREM

7.32

3SAT is polynomial time reducible to CLIQUE.

PROOF IDEA The polynomial time reduction f that we demonstrate from *3SAT* to *CLIQUE* converts formulas to graphs. In the constructed graphs, cliques of a specified size correspond to satisfying assignments of the formula. Structures within the graph are designed to mimic the behavior of the variables and clauses.

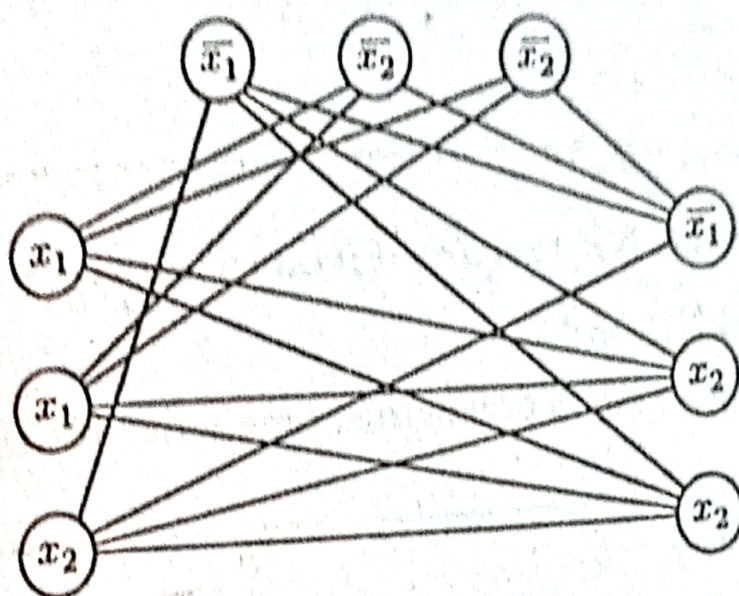
PROOF Let ϕ be a formula with k clauses such as

$$\phi = (a_1 \vee b_1 \vee c_1) \wedge (a_2 \vee b_2 \vee c_2) \wedge \dots \wedge (a_k \vee b_k \vee c_k).$$

The reduction f generates the string (G, k) , where G is an undirected graph defined as follows.

The nodes in G are organized into k groups of three nodes each called the *triples*, $t_1, \dots, t_k$. Each triple corresponds to one of the clauses in ϕ , and each node in a triple corresponds to a literal in the associated clause. Label each node of G with its corresponding literal in ϕ .

The edges of G connect all but two types of pairs of nodes in G . No edge is present between nodes in the same triple, and no edge is present between two nodes with contradictory labels, as in x_i and $\neg x_i$. Figure 7.33 illustrates this construction when $\phi = (x_1 \vee x_2 \vee x_3) \wedge (\neg x_1 \vee \neg x_2 \vee \neg x_3) \wedge (\neg x_1 \vee x_2 \vee x_3)$.

**FIGURE 7.33**

The graph that the reduction produces from
 $\phi = (x_1 \vee x_1 \vee x_2) \wedge (\bar{x}_1 \vee \bar{x}_2 \vee \bar{x}_2) \wedge (\bar{x}_1 \vee x_2 \vee x_2)$

Now we demonstrate why this construction works. We show that ϕ is satisfiable iff G has a k -clique.

Suppose that ϕ has a satisfying assignment. In that satisfying assignment, at least one literal is true in every clause. In each triple of G , we select one node corresponding to a true literal in the satisfying assignment. If more than one literal is true in a clause, we choose one of the true literals arbitrarily. The number of nodes selected is k .

times. When constructing a polynomial time reduction from 3SAT to a language, we sometimes look for structures in that language that can simulate the variables and clauses in Boolean formulas. Such structures are sometimes called *gadgets*. For example, in the reduction from 3SAT to CLIQUE presented in Theorem 7.32, individual nodes simulate variables and triples of nodes simulate clauses. An individual node may or may not be a member of the clique, corresponding to a variable that may or may not be true in a satisfying assignment. Each clause must contain a literal that is assigned TRUE. Correspondingly, each triple must contain a node in the clique (in order to reach the target size). The following corollary states that CLIQUE is NP-complete.

COROLLARY

7.43

CLIQUE is NP-complete.

THE VERTEX COVER PROBLEM

If G is an undirected graph, a *vertex cover* of G is a subset of the nodes where every edge of G touches one of those nodes. The vertex cover problem asks whether a graph contains a vertex cover of a specified size.

$VERTEX-COVER = \{ \langle G, k \rangle \mid G \text{ is an undirected graph that has a } k\text{-node vertex cover} \}$.

THEOREM 7.44

$VERTEX-COVER$ is NP-complete.

PROOF IDEA To show that $VERTEX-COVER$ is NP-complete, we must show that it is in NP and that all NP-problems are polynomial time reducible to it. The first part is easy; a certificate is simply a vertex cover of size k . To prove the second part, we show that $3SAT$ is polynomial time reducible to $VERTEX-COVER$. The reduction converts a 3cnf-formula ϕ into a graph G and a number k , so that ϕ is satisfiable whenever G has a vertex cover with k nodes. The conversion is done without knowing whether ϕ is satisfiable. In effect, G simulates ϕ . The graph contains gadgets that mimic the variables and clauses of the formula. Designing these gadgets requires a bit of ingenuity.

For the variable gadget, we look for a structure in G that can participate in the vertex cover in either of two possible ways, corresponding to the two possible truth assignments to the variable. The variable gadget contains two nodes connected by an edge. That structure works because one of these nodes must appear in the vertex cover. We arbitrarily associate TRUE and FALSE with these two nodes.

For the clause gadget, we look for a structure that induces the vertex cover to include nodes in the variable gadgets corresponding to at least one true literal in the clause. The gadget contains three nodes and additional edges so that any vertex cover must include at least two of the nodes, or possibly all three. Only two nodes would be required if one of the variable gadget nodes helps by covering an edge, as would happen if the associated literal satisfies that clause. Otherwise, three nodes would be required. Finally, we chose k so that the sought-after vertex cover has one node per variable gadget and two nodes per clause gadget.

PROOF Here are the details of a reduction from $3SAT$ to $VERTEX-COVER$ that operates in polynomial time. The reduction maps a Boolean formula ϕ to a graph G and a value k . For each variable x in ϕ , we produce an edge connecting two nodes. We label the two nodes in this gadget x and $\bar{x}$. Setting x to be TRUE corresponds to selecting the node labeled x for the vertex cover, whereas FALSE corresponds to the node labeled $\bar{x}$.

The gadgets for the clauses are a bit more complex. Each clause gadget is a triple of nodes that are labeled with the three literals of the clause. These three nodes are connected to each other and to the nodes in the variable gadgets that have the identical labels. Thus, the total number of nodes that appear in G is $2m + 3l$, where ϕ has m variables and l clauses. Let k be $m + 2l$. For example, if $\phi = (x_1 \vee x_1 \vee x_2) \wedge (\bar{x}_1 \vee \bar{x}_2 \vee \bar{x}_2) \wedge (\bar{x}_1 \vee x_2 \vee x_2)$, the reduction produces $\langle G, k \rangle$ from ϕ , where $k = 8$ and G takes the form shown in the following figure.

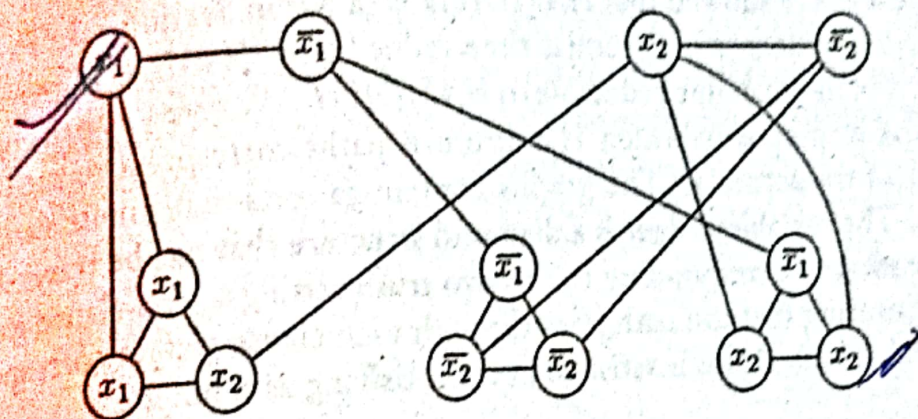


FIGURE 7.45

The graph that the reduction produces from $\phi = (x_1 \vee x_1 \vee x_2) \wedge (\bar{x}_1 \vee \bar{x}_2 \vee \bar{x}_2) \wedge (\bar{x}_1 \vee x_2 \vee x_2)$

To prove that this reduction works, we need to show that ϕ is satisfiable if and only if G has a vertex cover with k nodes. We start with a satisfying assignment. We first put the nodes of the variable gadgets that correspond to the true literals in the assignment into the vertex cover. Then, we select one true literal in every clause and put the remaining two nodes from every clause gadget into the vertex cover. Now we have a total of k nodes. They cover all edges because every variable gadget edge is clearly covered, all three edges within every clause gadget are covered, and all edges between variable and clause gadgets are covered. Hence G has a vertex cover with k nodes.

Second, if G has a vertex cover with k nodes, we show that ϕ is satisfiable by constructing the satisfying assignment. The vertex cover must contain one node in each variable gadget and two in every clause gadget in order to cover the edges of the variable gadgets and the three edges within the clause gadgets. That accounts for all the nodes, so none are left over. We take the nodes of the variable gadgets that are in the vertex cover and assign TRUE to the corresponding literals. That assignment satisfies ϕ because each of the three edges connecting the variable gadgets with each clause gadget is covered, and only two nodes of the clause gadget are in the vertex cover. Therefore, one of the edges must be covered by a node from a variable gadget and so that assignment satisfies the

THE HAMILTONIAN PATH PROBLEM

Recall that the Hamiltonian path problem asks whether the input graph contains a path from s to t that goes through every node exactly once.

THEOREM

7.46

HAMPATH is NP-complete.

PROOF IDEA We showed that HAMPATH is in NP in Section 7.3. To show that every NP-problem is polynomial time reducible to HAMPATH, we show that 3SAT is polynomial time reducible to HAMPATH. We give a way to convert 3cnf-formulas to graphs in which Hamiltonian paths correspond to satisfying assignments of the formula. The graphs contain gadgets that mimic variables and clauses. The variable gadget is a diamond structure that can be traversed in either of two ways, corresponding to the two truth settings. The clause gadget is a node. Ensuring that the path goes through each clause gadget corresponds to ensuring that each clause is satisfied in the satisfying assignment.

PROOF We previously demonstrated that HAMPATH is in NP, so all that remains to be done is to show $3SAT \leq_P HAMPATH$. For each 3cnf-formula ϕ , we show how to construct a directed graph G with two nodes, s and t , where a Hamiltonian path exists between s and t iff ϕ is satisfiable.

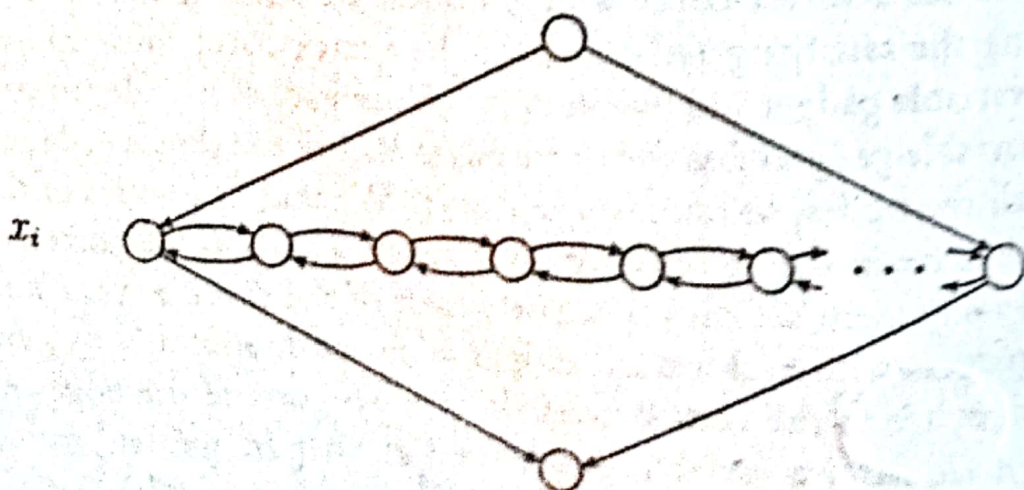
We start the construction with a 3cnf-formula ϕ containing k clauses,

$$\phi = (a_1 \vee b_1 \vee c_1) \wedge (a_2 \vee b_2 \vee c_2) \wedge \dots \wedge (a_k \vee b_k \vee c_k),$$

where each a , b , and c is a literal x_i or $\bar{x}_i$. Let $x_1, \dots, x_l$ be the l variables of ϕ .

Now we show how to convert ϕ to a graph G . The graph G that we construct has various parts to represent the variables and clauses that appear in ϕ .

We represent each variable x_i with a diamond-shaped structure that contains a horizontal row of nodes, as shown in the following figure. Later we specify the number of nodes that appear in the horizontal row.



Next, we consider an undirected version of the Hamiltonian path problem, called *UHAMPATH*. To show that *UHAMPATH* is NP-complete, we give a polynomial time reduction from the directed version of the problem.

THEOREM

7.55

UHAMPATH is NP-complete.

PROOF The reduction takes a directed graph G with nodes s and t , and constructs an undirected graph G' with nodes s' and t' . Graph G has a Hamiltonian path from s to t iff G' has a Hamiltonian path from s' to t' . We describe G' as follows.

Each node u of G , except for s and t , is replaced by a triple of nodes u^{in} , u^{mid} , and u^{out} in G' . Nodes s and t in G are replaced by nodes $s^{\text{out}} = s'$ and $t^{\text{in}} = t'$ in G' . Edges of two types appear in G' . First, edges connect u^{mid} with u^{in} and u^{out} . Second, an edge connects u^{out} with v^{in} if an edge goes from u to v in G . That completes the construction of G' .

We can demonstrate that this construction works by showing that G has a Hamiltonian path from s to t iff G' has a Hamiltonian path from s^{out} to t^{in} . To show one direction, we observe that a Hamiltonian path P in G ,

$$s, u_1, u_2, \dots, u_k, t,$$

has a corresponding Hamiltonian path P' in G' ,

$$s^{\text{out}}, u_1^{\text{in}}, u_1^{\text{mid}}, u_1^{\text{out}}, u_2^{\text{in}}, u_2^{\text{mid}}, u_2^{\text{out}}, \dots, t^{\text{in}}.$$

To show the other direction, we claim that any Hamiltonian path in G' from s^{out} to t^{in} must go from a triple of nodes to a triple of nodes, except for the start and finish, as does the path P' we just described. That would complete the proof because any such path has a corresponding Hamiltonian path in G . We prove the claim by following the path starting at node s^{out} . Observe that the next node in the path must be u_i^{in} for some i because only those nodes are connected to s^{out} . The next node must be u_i^{mid} because no other way is available to include u_i^{mid} in the Hamiltonian path. After u_i^{mid} comes u_i^{out} because that is the only other node to which u_i^{mid} is connected. The next node must be u_j^{in} for some j because no other available node is connected to u_i^{out} . The argument then repeats until t^{in} is reached.

THE SUBSET SUM PROBLEM

Recall the *SUBSET-SUM* problem defined on page 297. In that problem, we were given a collection of numbers $x_1, \dots, x_k$ together with a target number t , and were to determine whether the collection contains a subcollection that adds up to t . We now show that this problem is NP-complete.

THEOREM

7.56

SUBSET-SUM is NP-complete.

PROOF IDEA We have already shown that *SUBSET-SUM* is in NP in Theorem 7.25. We prove that all languages in NP are polynomial time reducible to *SUBSET-SUM* by reducing the NP-complete language *3SAT* to it. Given a 3cnf-formula ϕ , we construct an instance of the *SUBSET-SUM* problem that contains a subcollection summing to the target t if and only if ϕ is satisfiable. Call this subcollection T .

To achieve this reduction, we find structures of the *SUBSET-SUM* problem that represent variables and clauses. The *SUBSET-SUM* problem instance that we construct contains numbers of large magnitude presented in decimal notation. We represent variables by pairs of numbers and clauses by certain positions in the decimal representations of the numbers.

We represent variable x_i by two numbers, y_i and z_i . We prove that either y_i or z_i must be in T for each i , which establishes the encoding for the truth value of x_i in the satisfying assignment.

Each clause position contains a certain value in the target t , which imposes a requirement on the subset T . We prove that this requirement is the same as the one in the corresponding clause—namely, that one of the literals in that clause is assigned TRUE.

PROOF We already know that *SUBSET-SUM* $\in$ NP, so we now show that *3SAT* $\leq_P$ *SUBSET-SUM*.

Let ϕ be a Boolean formula with variables $x_1, \dots, x_l$ and clauses $c_1, \dots, c_k$. The reduction converts ϕ to an instance of the *SUBSET-SUM* problem $\langle S, t \rangle$, wherein the elements of S and the number t are the rows in the table in Figure 7.57, expressed in ordinary decimal notation. The rows above the double line are labeled

$$y_1, z_1, y_2, z_2, \dots, y_l, z_l \quad \text{and} \quad g_1, h_1, g_2, h_2, \dots, g_k, h_k$$

and constitute the elements of S . The row below the double line is t .

Thus, S contains one pair of numbers, y_i, z_i , for each variable x_i in ϕ . The decimal representation of these numbers is in two parts, as indicated in the table. The left-hand part comprises a 1 followed by $l - i$ 0s. The right-hand part contains one digit for each clause, where the digit of y_i in column c_j is 1 if clause c_j contains literal x_i , and the digit of z_i in column c_j is 1 if clause c_j contains literal $\bar{x}_i$. Digits not specified to be 1 are 0.

The table is partially filled in to illustrate sample clauses, c_1, c_2 , and c_k :

$$(x_1 \vee \bar{x}_2 \vee x_3) \wedge (x_2 \vee x_3 \vee \dots) \wedge \dots \wedge (\bar{x}_3 \vee \dots \vee \dots).$$

Additionally, S contains one pair of numbers, g_j, h_j , for each clause c_j . These two numbers are equal and consist of a 1 followed by $k - j$ 0s.